Prompt para o GitHub Copilot — Módulo WhatsApp (WAHA) no Osteogest

Como usar este documento

1. Abrir o repositório do **Osteogest** no VS Code.
2. Criar um branch dedicado: `git checkout -b feature/whatsapp-module`.
3. Abrir o **Copilot Chat** em modo **Agent**.
4. Copiar todo o texto entre **INÍCIO DA PROMPT** e **FIM DA PROMPT** e colar no chat.
5. O Copilot começa pela **Fase 0 (análise)** e pára à espera de aprovação. Rever o plano e responder "aprovado, avança para a Fase 1".
6. Repetir fase a fase. No fim de cada fase: correr testes/build, rever o diff e fazer commit.

> Não aprovar fases seguidas sem testar. Cada fase deve deixar a aplicação a funcionar e com o módulo desligado por omissão.

Pré-requisitos (ver o "Guia de Deploy"):

- Stack `waha` a correr no Portainer, na rede Docker `waha_net`.
- Stack do Osteogest ligada à rede `waha_net`, com `WHATSAPP_MODULE_ENABLED` e `WHATSAPP_ENCRYPTION_KEY`.
- Clínica provisionada no WAHA: **nome da sessão** + **chave de API restrita a essa sessão** (`provision-clinic.sh`).

INÍCIO DA PROMPT

Papel e objetivo

És um engenheiro sénior de Next.js/TypeScript. Vais criar no Osteogest um **módulo WhatsApp independente** que integra com um servidor **WAHA (WhatsApp HTTP API)** já existente, para enviar mensagens WhatsApp aos pacientes a partir do número da clínica, acompanhar o estado de entrega e receber respostas.

Requisitos principais:

1. **Sem regressões**: o módulo é isolado, aditivo, desligado por omissão e removível sem afetar o resto da aplicação.
2. **Configuração no backoffice**: toda a configuração WhatsApp (URL do WAHA, sessão, chave de API, número, webhook) é gerida por administradores numa página do Osteogest — **não** em variáveis de ambiente.
3. **Multi-deployment isolado**: o Osteogest vai ser instalado para várias clínicas; cada deployment tem a sua base de dados, o seu número WhatsApp, a sua sessão WAHA, a sua chave de API e o seu segredo de webhook. Nada é partilhado nem fixo no código (nunca hardcode de nomes de sessão, números ou URLs).

Stack do Osteogest (confirma no repositório)

- Next.js 16 (App Router; rotas API em `app/api/**/route.ts`; em Next 16 o middleware chama-se `proxy.ts` — confirma qual existe)
- React 19 + Tailwind CSS 4

- TypeScript 5
- Prisma 6 + MySQL 8.4
- Autenticação JWT com `jose` + `bcryptjs`
- Validação com `zod`
- Deploy em Docker/Portainer atrás de nginx-proxy-manager (container `osteogest`, porta interna 3000)

Como o WAHA está organizado (contexto)

- Um servidor WAHA pode servir várias clínicas. Cada clínica = **uma sessão** (um número WhatsApp).
- O administrador do WAHA cria, para cada clínica, a sessão e uma **chave de API restrita a essa sessão** (permissões `read`, `send`, `control`, `setting`). Essa chave:
  - pode: ver o estado da sua sessão, obter QR/código de emparelhamento, iniciar/parar/reiniciar/terminar sessão, alterar a configuração da sessão (webhooks), enviar mensagens, verificar números, resolver LIDs;
  - **não pode**: criar ou apagar sessões, gerir chaves, nem aceder a outras sessões (responde **403**).
- **Não existe webhook global**: cada sessão tem o seu próprio webhook (URL + segredo HMAC), que o **Osteogest configura** na sua sessão através da API.
- O Osteogest **nunca** recebe a chave admin do WAHA.

Regras de isolamento do código (obrigatórias)

1. **Todo o código novo** numa única pasta: `src/modules/whatsapp/` (ou `modules/whatsapp/` se o projeto não usar `src/`). Segue a convenção de pastas existente.
2. As pastas `app/` só recebem **ficheiros finos** (route handlers e pages) que importam do módulo e não contêm lógica.
3. **Não alterar** modelos Prisma existentes, tabelas existentes, componentes existentes, testes existentes nem versões de dependências.
4. **Sem relações Prisma** para modelos existentes: `patientId`, `appointmentId` e `userId` são colunas simples com índice (sem `@relation`).
5. **Migração apenas aditiva**: só `CREATE TABLE` / `CREATE INDEX` de tabelas com prefixo `whatsapp_`. Mostra-me o SQL antes de aplicar. Proibido `ALTER`/`DROP` em tabelas existentes.
6. **Um único adaptador** (`server/osteogest-adapter.ts`) é o único ficheiro que conhece modelos e helpers existentes (pacientes, consultas, utilizadores, roles, autenticação, notificações).
7. **Feature flag** `WHATSAPP_MODULE_ENABLED` (default `false`): desligado → rotas do módulo respondem 404, UI escondida, nenhum worker arranca, aplicação igual à atual.
8. **Pontos de integração permitidos** em ficheiros existentes (cada alteração mínima e listada no resumo final):
   - entrada no menu/navegação (só com flag ligada e para quem tiver permissão);
   - entrada "WhatsApp" na área de configurações/backoffice existente;
   - slot `<WhatsappPatientPanel patientId={...} />` na ficha do paciente;
   - botão `<WhatsappReminderButton appointmentId={...} />` na consulta/agenda;
   - exclusão de `/api/webhooks/waha` da autenticação JWT em `proxy.ts`/`middleware.ts`;
   - registo do worker em `instrumentation.ts` (criar se não existir);
   - `.env.example` e `docker-compose` (rede externa `waha_net` + 2 variáveis).
   Qualquer outra alteração fora do módulo requer a minha aprovação explícita.
9. Novas dependências: apenas `libphonenumber-js` (e `server-only`, se necessário). Pedir aprovação para outras.
10. Seguir o estilo do projeto (lint, formatação, nomes, componentes). Não refatorar código existente.

Estrutura do módulo

```
src/modules/whatsapp/
  index.ts                    # exports públicos (componentes + tipos) — nada server-only
  env.ts                      # WHATSAPP_MODULE_ENABLED, WHATSAPP_ENCRYPTION_KEY (zod)
  server/
    crypto.ts                 # AES-256-GCM (cifrar/decifrar segredos na BD)
    settings.ts               # ler/gravar configurações (cache 30 s, invalidar ao gravar)
    waha-client.ts            # cliente HTTP do WAHA, criado a partir das configurações
    waha-types.ts             # schemas zod das respostas/eventos
    phone.ts                  # normalização de números -> chatId
    hmac.ts                   # verificação da assinatura dos webhooks
    templates.ts              # render de templates
    service.ts                # casos de uso
    webhook-setup.ts          # aplicar/verificar webhook da sessão no WAHA
    webhook-handler.ts        # processamento de eventos recebidos
    outbox-worker.ts          # fila de envio sequencial + lembretes automáticos
    repository.ts             # Prisma (tabelas whatsapp_*)
    osteogest-adapter.ts      # ÚNICO ponto de contacto com o core do Osteogest
    guards.ts                 # requireModuleEnabled(), requireUser(), requireAdmin()
  api/                        # handlers usados pelos route.ts finos
  components/
    WhatsappSettingsPage.tsx
    WhatsappConnectionForm.tsx
    WhatsappSessionCard.tsx
    WhatsappPairingDialog.tsx
    WhatsappTemplatesEditor.tsx
    WhatsappPatientPanel.tsx
    WhatsappSendDialog.tsx
    WhatsappMessageHistory.tsx
    WhatsappReminderButton.tsx
    WhatsappBulkRemindersDialog.tsx
  __tests__/
```

Variáveis de ambiente (apenas duas)

Variável	Exemplo	Notas
`WHATSAPP_MODULE_ENABLED`	`false`	Feature flag
`WHATSAPP_ENCRYPTION_KEY`	64 caracteres hex (32 bytes)	Chave AES-256-GCM para cifrar os segredos guardados na BD

Com a flag ligada e sem `WHATSAPP_ENCRYPTION_KEY` válida: registar um erro claro e manter o módulo inativo (sem derrubar a aplicação). Todas as outras configurações vêm do backoffice.

Cifragem de segredos (`crypto.ts`)

- AES-256-GCM com IV aleatório de 12 bytes por valor; guardar `v1:<iv b64>:<tag b64>:<ciphertext b64>`.
- Segredos cifrados: **chave de API do WAHA** e **segredo HMAC do webhook**.
- Nunca devolver segredos ao browser. A UI mostra apenas "definida" + os **últimos 4 caracteres** da chave.
- Se a decifragem falhar (chave de ambiente mudou): estado "Configuração inválida — volte a introduzir a chave de API" e bloquear envios.

Modelos Prisma (novos, prefixo `whatsapp_`)

Se o projeto usar schema em vários ficheiros (`prisma/schema/`), criar `prisma/schema/whatsapp.prisma`; caso

contrário, acrescentar um bloco delimitado no fim de `schema.prisma`. Usar `@@map("whatsapp_...")`.

- **WhatsappSettings** (registo único, id fixo `1`):
  - ligação: `wahaBaseUrl` (default `"http://waha:3000"`), `sessionName`, `apiKeyEncrypted?`, `apiKeyLast4?`;
  - webhook: `webhookCallbackUrl` (default `"http://osteogest:3000/api/webhooks/waha"`), `webhookSecretEncrypted?`, `webhookAppliedAt?`, `webhookConfigHash?`;
  - clínica: `clinicPhoneNumber` (dígitos E.164 sem "+"), `defaultCountry` (default `"PT"`);
  - envio: `sendDelayMs` (default `4000`), `remindersEnabled` (default `false`), `reminderHour` (default `18`), `reminderDaysBefore` (default `1`), `reminderTemplateKey?`;
  - `enabled` (bool — interruptor funcional no backoffice, além da flag de ambiente), `updatedByUserId?`, `updatedAt`.
- **WhatsappSettingsAudit**: `id`, `userId`, `field`, `oldValueMasked`, `newValueMasked`, `createdAt` (nunca guardar segredos em claro).
- **WhatsappTemplate**: `id`, `key` (único), `name`, `body` (text), `active`, timestamps. Seeds: `appointment_reminder`, `appointment_confirmation`, `free_text`.
- **WhatsappConsent**: `patientId` (único), `granted`, `grantedAt?`, `revokedAt?`, `userId?`, `source` (`"ficha"` | `"resposta_whatsapp"`).
- **WhatsappMessage**: `id`, `patientId?`, `appointmentId?`, `direction` (`OUTBOUND`|`INBOUND`), `chatId`, `phoneMasked`, `wahaMessageId?` (único), `type` (`TEXT`|`FILE`), `body` (text), `fileName?`, `status` (`QUEUED`|`SENDING`|`PENDING`|`SERVER`|`DEVICE`|`READ`|`PLAYED`|`ERROR`|`RECEIVED`), `error?`, `attempts`, `scheduledAt?`, `sentAt?`, `ackAt?`, `sentByUserId?`, timestamps. Índices em `patientId`, `appointmentId`, `status`, `createdAt`.
- **WhatsappWebhookEvent**: `id`, `requestId` (único; header `X-Webhook-Request-Id`), `event`, `session`, `payload` (Json), `processedAt?`, `error?`, `receivedAt`. Limpar registos com mais de 30 dias.
- **WhatsappSessionState** (registo único): `status`, `meId?`, `pushName?`, `lastEventAt`.

Referência da API do WAHA (confirmada no código e testada)

Base: `settings.wahaBaseUrl`. Todos os pedidos com `X-Api-Key: <chave da clínica>` e `Content-Type: application/json`. `{s}` = `settings.sessionName`. Timeout de 15 s por pedido.

Operação	Pedido	Resposta
Estado + configuração da sessão	`GET /api/sessions/{s}`	`{ name, status, config: { metadata?, webhooks? }, me?: { id, pushName } }`
Atualizar configuração da sessão	`PUT /api/sessions/{s}` body `{ config: {...} }`	sessão. **Substitui toda a `config` e reinicia a sessão** se estiver a correr
Iniciar / parar / reiniciar	`POST /api/sessions/{s}/start` · `/stop` · `/restart`	sessão
Terminar sessão (desligar número)	`POST /api/sessions/{s}/logout`	sessão
QR code (PNG)	`GET /api/{s}/auth/qr?format=image` com `Accept: image/png`	PNG
Código de emparelhamento	`POST /api/{s}/auth/request-code` body `{ "phoneNumber": "351912345678" }`	`{ code }`
Verificar número	`GET /api/contacts/check-exists?phone=351912345678&session={s}`	`{ numberExists, chatId }`
Enviar texto	`POST /api/sendText` body `{ session, chatId, text,`	mensagem com `id`

Operação	Pedido	Resposta
	`linkPreview: false }`	
Enviar ficheiro	`POST /api/sendFile` body `{ session, chatId, caption, file: { mimetype, filename, data: "<base64>" } }`	mensagem com `id`
Enviar imagem	`POST /api/sendImage` (formato igual a `sendFile`)	mensagem com `id`
LID → telefone	`GET /api/{s}/lids/{lid}` (URL-encode do `@`)	`{ lid, pn }`

A chave da clínica **não** permite `POST /api/sessions` (criar), `DELETE /api/sessions/{s}` nem `/api/keys` → o módulo **não** deve tentar criar ou apagar sessões. Se a sessão não existir (404), mostrar: "Sessão não encontrada no WAHA — peça ao administrador para provisionar a clínica".

Estados da sessão: `STOPPED`, `STARTING`, `SCAN_QR_CODE`, `PASSKEY_REQUIRED`, `PASSKEY_CONFIRMATION_REQUIRED`, `WORKING`, `FAILED`. Só enviar quando `WORKING`.

chatId: `<dígitos E.164 sem +>@c.us` (ex.: `351912345678@c.us`). Grupos `@g.us`, estados `status@broadcast`, canais `@newsletter`; contactos podem aparecer como `<n>@lid`.

Mapeamento de erros HTTP (mensagens em português na UI):

HTTP	Significado	Mensagem
401	Chave inválida, revogada ou inativa	"Chave de API inválida ou revogada"
403	Chave de outra sessão / sem permissão	"A chave não tem acesso a esta sessão — confirme o nome da sessão"
404	Sessão inexistente	"Sessão não encontrada no WAHA"
422/500	Sessão não pronta ou pedido inválido	"Não foi possível enviar — verifique o estado da ligação"
timeout / rede	WAHA inacessível	"Servidor WhatsApp inacessível"

Configurar o webhook da sessão (`webhook-setup.ts`)

Ação **"Aplicar configuração de webhooks"** (e automaticamente após guardar alterações de ligação, se a sessão estiver acessível):

1. Se ainda não existir, gerar o segredo HMAC (`crypto.randomBytes(32).toString("hex")`) e gravá-lo cifrado.
2. `GET /api/sessions/{s}` e ler a `config` atual.
3. Construir a nova `config` **preservando todos os campos existentes** (`metadata`, `proxy`, etc.) e substituindo apenas, em `webhooks`, a entrada cujo `url` = `webhookCallbackUrl` (ou acrescentá-la):

```
{
  "url": "<webhookCallbackUrl>",
  "events": ["session.status", "message", "message.ack"],
  "hmac": { "key": "<segredo>" },
  "retries": { "policy": "exponential", "delaySeconds": 2, "attempts": 10 }
}
```

4. Calcular um hash (sha256) da entrada; se for igual a `webhookConfigHash` e a entrada já existir no WAHA → não fazer nada (evita reinícios desnecessários).
5. Caso contrário: avisar na UI que "a sessão vai reiniciar alguns segundos", fazer `PUT /api/sessions/{s}`

com a config completa e gravar `webhookAppliedAt` e `webhookConfigHash`.
6. Ação **"Regenerar segredo do webhook"**: novo segredo → reaplicar.
7. No cartão de estado, mostrar "Webhook configurado" / "Webhook em falta ou desatualizado" (comparando com o que o WAHA devolve).

Webhooks recebidos (`POST /api/webhooks/waha`, `runtime = "nodejs"`)

Cabeçalhos enviados pelo WAHA: `X-Webhook-Hmac` (HMAC-SHA512 **hex** do corpo raw), `X-Webhook-Hmac-Algorithm` (`sha512`), `X-Webhook-Request-Id`, `X-Webhook-Timestamp` (ms).

Corpo: `{ id, timestamp, session, engine, me, environment, event, payload }`.

Payloads:

- `session.status` → `{ name, status, statuses: [{ status, timestamp }], data? }`
- `message.ack` → `{ id, from, to, participant, fromMe, ack, ackName }`, com `ack`: `-1` ERROR, `0` PENDING, `1` SERVER, `2` DEVICE, `3` READ, `4` PLAYED
- `message` → `{ id, timestamp, from, fromMe, source, to, body, hasMedia, media?: { url, mimetype, filename } }`

Regras do handler:

1. Flag desligada ou `settings.enabled=false` → 404.
2. Ler o corpo com `await req.text()` **antes** de qualquer parse.
3. Decifrar o segredo HMAC das configurações; calcular HMAC-SHA512 hex e comparar com `crypto.timingSafeEqual` (buffers do mesmo tamanho). Falha → 401 e log de aviso (sem corpo).
4. `requestId` já existente em `WhatsappWebhookEvent` → 200 sem reprocessar.
5. `session` ≠ `settings.sessionName` → 200 e ignorar (log de aviso).
6. Gravar o evento, processar e responder **200 rapidamente**; erros de processamento ficam em `WhatsappWebhookEvent.error`, mas a resposta é 200.
7. `session.status` → atualizar `WhatsappSessionState`; em `FAILED`, `STOPPED` ou `SCAN_QR_CODE`, notificar os administradores (via adaptador; se não houver mecanismo, banner na página de configurações).
8. `message.ack` → atualizar `WhatsappMessage` por `wahaMessageId`, **sem regredir** o estado (PENDING < SERVER < DEVICE < READ < PLAYED; ERROR é final); gravar `ackAt`.
9. `message` com `fromMe=false`:
   - `@c.us` → número = dígitos antes do `@`; `@lid` → `GET /api/{s}/lids/{lid}` e usar `pn`; ignorar `@g.us`, `status@broadcast`, `@newsletter`;
   - encontrar o paciente pelo número (via adaptador); gravar INBOUND `RECEIVED` (com `patientId` nulo se não houver correspondência);
   - resposta a um lembrete das últimas 72 h: `1`/`sim`/`confirmo` → confirmar consulta (via adaptador) e responder com o template de confirmação; `2`/`remarcar`/`não` → sinalizar a consulta para contacto;
   - `STOP`/`parar` → revogar consentimento (`source="resposta_whatsapp"`);
   - media recebida: guardar só metadados (o WAHA apaga ficheiros ao fim de 180 s).
10. `message` com `fromMe=true` e `source="APP"` (enviada no telemóvel) → gravar OUTBOUND sem utilizador.

Normalização de números (`phone.ts`)

- `libphonenumber-js`, país por omissão `settings.defaultCountry`.
- Aceitar `+351 912 345 678`, `00351912345678`, `912345678`, `912-345-678`.
- Devolver `{ e164Digits, chatId, masked }` (ex.: `351912345678`, `351912345678@c.us`, `351*****678`) ou erro de validação.
- Antes do **primeiro** envio a um número: `check-exists`; se `numberExists=false` → "Este número não tem

WhatsApp". Cache do `chatId` por 30 dias.

Rotas API (ficheiros finos em `app/api/...`)

Todas: `runtime = "nodejs"`, `dynamic = "force-dynamic"`, `requireModuleEnabled()`, zod no input, erros `{ error: { code, message } }`, `Cache-Control: no-store`.

Método e rota	Permissão	Descrição
`GET /api/whatsapp/settings`	admin	Configurações sem segredos: `{ ..., apiKeyConfigured, apiKeyLast4, webhookSecretConfigured, webhookAppliedAt }`
`PUT /api/whatsapp/settings`	admin	Grava configurações; `apiKey` é **só de escrita** (vazio = manter atual); regista auditoria
`POST /api/whatsapp/settings/test`	admin	Testa URL + chave + sessão com `GET /api/sessions/{s}`; devolve `{ ok, httpStatus, sessionStatus, me, latencyMs, message }`
`POST /api/whatsapp/settings/apply-webhook`	admin	Aplica a configuração de webhook (ver secção acima)
`POST /api/whatsapp/settings/rotate-webhook-secret`	admin	Novo segredo HMAC + reaplicar
`GET /api/whatsapp/session`	autenticado	Estado, número ligado, aviso se `me.id` ≠ `clinicPhoneNumber`, estado do webhook
`POST /api/whatsapp/session/start`	admin	`POST /api/sessions/{s}/start`
`GET /api/whatsapp/session/qr`	admin	Proxy do PNG
`POST /api/whatsapp/session/pairing-code`	admin	Código para `clinicPhoneNumber`
`POST /api/whatsapp/session/restart`	admin	Reiniciar
`POST /api/whatsapp/session/logout`	admin	Desligar o número (exige `{ confirm: true }`)
`GET/POST /api/whatsapp/templates`	admin	Listar/criar
`PUT/DELETE /api/whatsapp/templates/[id]`	admin	Editar/desativar
`GET/PUT /api/whatsapp/consent/[patientId]`	autenticado	Consentimento
`POST /api/whatsapp/messages`	autenticado	Enviar: `{ patientId, appointmentId?, templateKey?, text?, file? }`
`GET /api/whatsapp/messages?patientId=&cursor=`	autenticado	Histórico paginado
`POST /api/whatsapp/reminders/`	autenticado	Consultas elegíveis para uma data

Método e rota	Permissão	Descrição
`preview`		
`POST /api/whatsapp/reminders/enqueue`	autenticado	Enfileirar lembretes (sem duplicar)
`POST /api/webhooks/waha`	pública (HMAC)	Webhooks; excluir da autenticação JWT

Validação das configurações (zod): `wahaBaseUrl` e `webhookCallbackUrl` com `http`/`https` e sem credenciais no URL; `sessionName` `^[a-zA-Z0-9_-]{1,64}$`; `apiKey` 16–128 caracteres; `clinicPhoneNumber` normalizado por `phone.ts`.

Regras de envio (`POST /api/whatsapp/messages`):

- 409 sem consentimento ativo, com `settings.enabled=false`, configuração incompleta ou sessão ≠ `WORKING`.
- Templates com `{{nome}}`, `{{primeiroNome}}`, `{{data}}` (dd/MM/yyyy), `{{hora}}` (HH:mm), `{{clinica}}`, `{{terapeuta}}`, `{{texto}}` (texto livre); fuso `Europe/Lisbon`; placeholders desconhecidos → erro.
- Ficheiros: PDF, JPEG ou PNG, até 10 MB; validar mimetype pelos magic bytes.
- Criar `WhatsappMessage` `QUEUED`; o **worker** envia (nunca em paralelo). Envios individuais processados de imediato.
- Rate limit: 20 envios/minuto por utilizador.

Worker de envio e lembretes (`outbox-worker.ts`)

- Registado em `instrumentation.ts` → `register()`, só com `NEXT_RUNTIME === "nodejs"` e flag ligada; em cada ciclo, ler `settings` (se `enabled=false` ou config incompleta, não fazer nada).
- A cada 5 s: pegar **uma** mensagem `QUEUED` com `scheduledAt <= now` (lock otimista `QUEUED → SENDING`), enviar, gravar `wahaMessageId` e `PENDING`/`SERVER`, ou `ERROR`.
- Entre envios: `sendDelayMs` + jitter de 0–3 s.
- Retries: até 3 para timeouts/5xx (30 s, 2 min, 10 min); 401/403/404/422 → `ERROR` imediato com a mensagem mapeada.
- No arranque: `SENDING` com mais de 5 min → `QUEUED`.
- Lembretes automáticos (`remindersEnabled`): uma vez por dia a partir de `reminderHour` (Europe/Lisbon), enfileirar lembretes das consultas daqui a `reminderDaysBefore` dias, de pacientes com consentimento e sem lembrete. Guardar a última execução.
- Parar de forma limpa em `SIGTERM`.

Interface (React 19 + Tailwind 4, seguindo os componentes existentes)

a) Backoffice → Configurações → Integrações → WhatsApp (admin), organizado em passos:

1. **Ligação ao WAHA**: URL do WAHA (interno), nome da sessão, chave de API (input password; placeholder "•••• 1a2b" quando definida; vazio = manter), botão **Testar ligação** com resultado claro (✓ ligado / chave inválida / sem acesso à sessão / sessão inexistente / servidor inacessível).
2. **Webhook**: URL de callback (interno, com valor por omissão), estado "configurado/desatualizado", botões **Aplicar configuração** e **Regenerar segredo** (com aviso de reinício da sessão).
3. **Número da clínica**: número WhatsApp, país; cartão de estado com badge (Ligado verde `WORKING`, A aguardar emparelhamento amarelo, Desligado cinzento, Erro vermelho), número e nome ligados, aviso se o número ligado ≠ configurado; botões **Ligar número**, **Reiniciar**, **Terminar sessão** (confirmação).
   - Diálogo **Ligar número**: separador QR code (imagem atualizada a cada 20 s, polling de estado a cada

3 s até `WORKING`) e separador Código (código em letra grande + instruções "WhatsApp → Definições → Dispositivos ligados → Ligar um dispositivo → Ligar com número de telefone").

4. **Envio e lembretes**: ativar módulo (`enabled`), intervalo entre envios, lembretes automáticos (ligar, hora, dias de antecedência, template).
5. **Templates**: lista, editor com placeholders e pré-visualização com dados fictícios.
6. **Auditoria**: últimas alterações às configurações (quem, quando, campo; valores mascarados).

b) Ficha do paciente (`<WhatsappPatientPanel />`): consentimento (data + utilizador); botão **Enviar WhatsApp** (desativado com tooltip se faltar consentimento, número válido ou ligação); diálogo com template ou texto livre, pré-visualização, anexo opcional; histórico em bolhas com estados: relógio (QUEUED/SENDING/PENDING), ✓ (SERVER), ✓✓ (DEVICE), ✓✓ azul (READ/PLAYED), ⚠ (ERROR com tooltip). Polling de 10 s enquanto visível.

c) Agenda / consultas: `<WhatsappReminderButton />` (enviar lembrete + estado do último) e `<WhatsappBulkRemindersDialog />` (data, lista de elegíveis com motivos de exclusão, confirmação, progresso).

Textos da UI em **português de Portugal**.

Segurança e RGPD

- Segredos (chave WAHA, HMAC) só no servidor, cifrados em repouso; `import "server-only"` em `server/*`; o browser só fala com rotas do Osteogest.
- Autorização em todas as rotas (helpers existentes via adaptador); configurações só para administradores.
- Validação zod, limites de tamanho, `Cache-Control: no-store`.
- **Logs sem conteúdo das mensagens**, sem segredos e com números mascarados.
- Templates sem dados clínicos; aviso na UI para não enviar diagnósticos ou notas clínicas.
- Auditoria de alterações de configuração, envios (`sentByUserId`) e consentimentos.

Testes e qualidade

- Testes unitários (framework existente; se não houver, propor Vitest e pedir aprovação):
  - `crypto.ts` (ida e volta, IV diferente por valor, chave errada falha);
  - `phone.ts` (formatos PT, internacionais, inválidos);
  - `hmac.ts` (válida, inválida, tamanho diferente, corpo alterado);
  - `webhook-setup.ts` (preserva `metadata`, substitui só a entrada do callback, não reaplica se o hash for igual);
  - mapeamento de erros HTTP (401/403/404/timeout) e não-regressão de ack;
  - templates (placeholders, fuso, desconhecidos);
  - `webhook-handler` (idempotência, sessão diferente, `@lid`, grupos ignorados, "1", "STOP");
  - `waha-client` com `fetch` mockado.
- No fim de **cada fase**: `tsc --noEmit`, lint, testes existentes + novos, `next build` — tudo a passar.
- Confirmar que com `WHATSAPP_MODULE_ENABLED=false` a aplicação se comporta como antes.

Plano por fases (parar no fim de cada fase e esperar aprovação)

Fase 0 — Análise (sem alterar código): estrutura do projeto, `app/`, layout, navegação e área de configurações/backoffice, helpers de autenticação/JWT e roles, `proxy.ts`/`middleware.ts`, `instrumentation.ts`, schema Prisma (single ou multi-file), modelos Patient/Appointment/User (nome, telefone, datas, estado da

consulta, terapeuta), mecanismo de notificações, framework de testes, scripts do `package.json`. Entregar o plano, ficheiros a criar, a **lista exata** de pontos de integração (com diff previsto) e riscos.

Fase 1 — Fundações: `env.ts`, `crypto.ts`, `waha-client`, `phone`, `hmac`, modelos Prisma + migração aditiva (mostrar SQL), seeds, `settings.ts`, adaptador, guards, testes.

Fase 2 — Backoffice: rotas de settings (incl. test, apply-webhook, rotate), rotas de sessão, página de configurações (passos 1–6), entrada no menu/configurações.

Fase 3 — Envio: consentimento, rota de envio, worker, painel do paciente, botão de lembrete.

Fase 4 — Webhooks: `/api/webhooks/waha`, exclusão no `proxy.ts`/`middleware.ts`, ack, mensagens recebidas, confirmação/remarcação, STOP.

Fase 5 — Lembretes automáticos e em massa: agendamento diário, diálogo em massa, limpeza de eventos antigos.

Fase 6 — Entrega: `.env.example` (`WHATSAPP_MODULE_ENABLED`, `WHATSAPP_ENCRYPTION_KEY`), `docker-compose` do Osteogest (serviço na rede externa `waha_net`, mantendo as redes existentes), `src/modules/whatsapp/README.md` (setup, provisionamento pelo administrador do WAHA, configuração no backoffice, desligar/remover o módulo) e resumo final (ficheiros criados, ficheiros existentes alterados com justificação, rollback).

Critérios de aceitação

☐ Com a flag desligada nada muda; testes e build passam.
☐ Um admin introduz URL, sessão e chave no backoffice, **testa a ligação** e vê mensagens claras para chave inválida (401), sessão errada (403) e sessão inexistente (404).
☐ **Aplicar configuração de webhooks** regista o callback na sessão, preservando o resto da configuração, e não reinicia a sessão se nada mudou.
☐ O admin liga o número por QR ou código e vê `WORKING`.
☐ Envio de template a paciente com consentimento: ✓ → ✓✓ → ✓✓ azul.
☐ Envio bloqueado sem consentimento, com número inválido ou ligação em baixo, com mensagens claras.
☐ Webhook com HMAC inválido → 401; webhook repetido não é reprocessado; eventos de outra sessão são ignorados.
☐ "1" confirma a consulta; "STOP" revoga o consentimento.
☐ Lembretes em massa enviados um a um, com intervalo, sem duplicados.
☐ Nenhum segredo aparece em respostas da API, logs ou auditoria; a BD guarda-os cifrados.
☐ Dois deployments do Osteogest (duas BD) ligados ao mesmo WAHA com sessões e chaves diferentes funcionam sem interferência.
☐ Remover o módulo = apagar `src/modules/whatsapp/`, os route files finos, os pontos de integração listados e (opcionalmente) as tabelas `whatsapp_*`.

FIM DA PROMPT

Anexo — Referência rápida

Provisionar uma clínica no WAHA (administrador, no servidor):

```
cd ~/projetos_WAHA/deploy/portainer
./provision-clinic.sh create osteogest-demo http://192.168.200.6:3000
```

Exemplo de envio de texto (com a chave da clínica):

```
POST http://waha:3000/api/sendText
```

```
X-Api-Key: <chave da clínica>
Content-Type: application/json

{
  "session": "osteogest-demo",
  "chatId": "351912345678@c.us",
  "text": "Olá Maria, lembramos a sua consulta amanhã às 10:00. Responda 1 para confirmar ou 2 para
remarcar.",
  "linkPreview": false
}
```

Exemplo de configuração da sessão com webhook (`PUT /api/sessions/osteogest-demo`):

```
{
  "config": {
    "metadata": { "app": "osteogest", "clinic": "osteogest-demo" },
    "webhooks": [
      {
        "url": "http://osteogest:3000/api/webhooks/waha",
        "events": ["session.status", "message", "message.ack"],
        "hmac": { "key": "<segredo gerado pelo Osteogest>" },
        "retries": { "policy": "exponential", "delaySeconds": 2, "attempts": 10 }
      }
    ]
  }
}
```

Verificação HMAC (Node.js):

```
import { createHmac, timingSafeEqual } from "node:crypto";

export function isValidWahaSignature(rawBody: string, signature: string | null, key: string): boolean {
  if (!signature) return false;
  const expected = createHmac("sha512", key).update(rawBody).digest("hex");
  const a = Buffer.from(expected, "hex");
  const b = Buffer.from(signature, "hex");
  return a.length === b.length && timingSafeEqual(a, b);
}
```

Cifragem AES-256-GCM (Node.js):

```
import { createCipheriv, createDecipheriv, randomBytes } from "node:crypto";

const key = Buffer.from(process.env.WHATSAPP_ENCRYPTION_KEY!, "hex"); // 32 bytes

export function encrypt(plain: string): string {
  const iv = randomBytes(12);
  const cipher = createCipheriv("aes-256-gcm", key, iv);
  const data = Buffer.concat([cipher.update(plain, "utf8"), cipher.final()]);
  return ["v1", iv.toString("base64"), cipher.getAuthTag().toString("base64"),
data.toString("base64")].join(":");
}

export function decrypt(value: string): string {
  const [, iv, tag, data] = value.split(":");
  const decipher = createDecipheriv("aes-256-gcm", key, Buffer.from(iv, "base64"));
  decipher.setAuthTag(Buffer.from(tag, "base64"));
  return Buffer.concat([decipher.update(Buffer.from(data, "base64")),
decipher.final()]).toString("utf8");
}
```

Templates sugeridos:

- appointment_reminder: "Olá {{primeiroNome}}, lembramos a sua consulta de osteopatia na {{clinica}} no dia {{data}} às {{hora}}. Responda 1 para confirmar ou 2 para remarcar."

- appointment_confirmation: "Obrigado, {{primeiroNome}}! A sua consulta de {{data}} às {{hora}} está confirmada."
- free_text: "{{texto}}"